



Mapas em Tiles, Colisões e Representação do Ambiente

AULA 03 – PROJETO DE INTELIGÊNCIA ARTIFICIAL

Sistemas de Informação · Disciplina: Projeto de Inteligência Artificial

"Como o computador entende o cenário que nós enxergamos?"

Do Cenário Visual para os Dados

O jogador enxerga paredes, árvores, água e caminhos. O computador, no entanto, enxerga posições, valores e regras. Essa tradução é fundamental para qualquer sistema de jogo inteligente.

O que o jogador vê

Um cenário rico: personagens, obstáculos, terrenos variados e decorações visuais que criam a experiência do jogo.

O que o computador vê

```
0 0 0 1 1
0 0 0 1 1
0 2 2 2 0
0 0 0 0 0
```

Uma matriz onde cada número representa um tipo de tile: 0 = chão, 1 = parede, 2 = água.

 **Conceito-chave:** O mapa é uma estrutura de dados. Toda a riqueza visual do cenário precisa ter uma representação computacional equivalente.

O que é um Tile?

Um **tile** é uma pequena unidade gráfica — normalmente um quadrado de tamanho fixo — utilizada como bloco construtivo para criar cenários maiores. Em vez de desenhar cada cenário do zero, os desenvolvedores reutilizam tiles de uma **tileset** para montar ambientes completos.

 **Chão / Grama**

Terreno base transitável

 **Parede**

Obstáculo intransponível

 **Água**

Bloqueio com aparência diferente

 **Árvore**

Obstáculo decorativo

 **Pedra / Ponte**

Terrenos especiais

Vantagens: **economia de memória**, reutilização de assets, facilidade de edição, organização em grid e integração natural com sistemas de IA.

Tile Map: O Mapa como Grade

Um **Tile Map** é a organização de tiles em linhas e colunas formando uma grade. Cada célula da grade possui um endereço único identificado por **linha** e **coluna** — exatamente como os índices de uma matriz bidimensional estudada em programação.

Estrutura do Tile Map

Cada posição é acessada por:

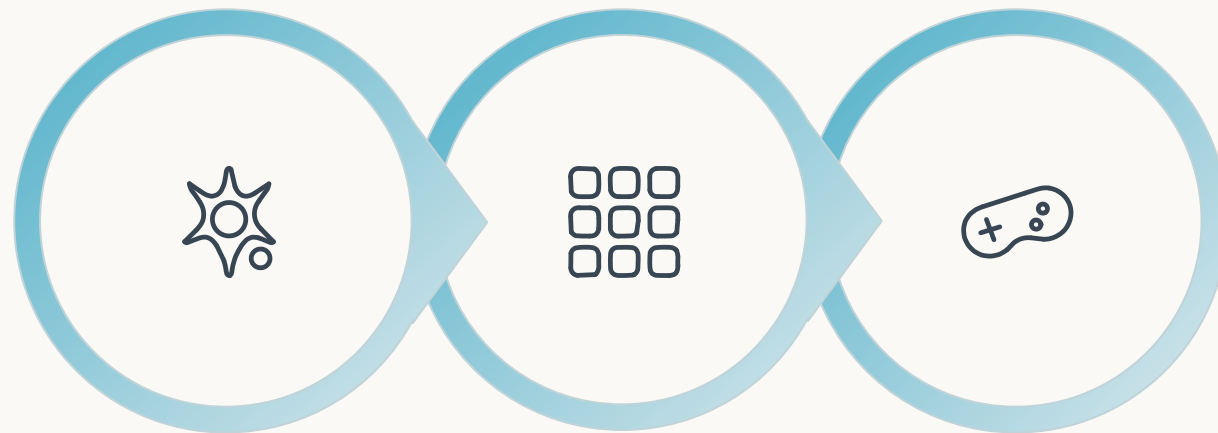
```
mapa[linha][coluna]  
mapa[row][col]
```

Ou equivalentemente por coordenadas **x, y** no espaço do grid.

Exemplo de acesso

```
mapa[0][0] → canto superior esq.  
mapa[2][3] → linha 2, coluna 3  
mapa[4][1] → linha 4, coluna 1
```

Esse endereçamento simples permite localizar qualquer tile instantaneamente — essencial para colisão e navegação de agentes.



Matriz

Tile Map

Cena

Representando o Mapa como Matriz

Matriz do mapa

```
0 0 0 0 0
0 1 1 1 0
0 1 0 0 0
0 1 0 2 2
0 0 0 0 0
```

Legenda:

0 = chão 1 = parede 2 = água

Um valor, múltiplos significados

Cada número na matriz pode representar simultaneamente:

- **Aparência:** qual sprite renderizar
- **Propriedade:** pode ou não ser atravessado
- **Comportamento:** causa dano, reduz velocidade
- **Custo:** quanto custa mover por esse tile

Essa riqueza semântica torna a matriz muito mais do que um mapa visual — ela se torna a **representação computacional do ambiente**.

Camada Visual × Camada Lógica

Todo ambiente de jogo possui pelo menos duas representações simultâneas e igualmente importantes.

Camada Visual

O que o jogador vê

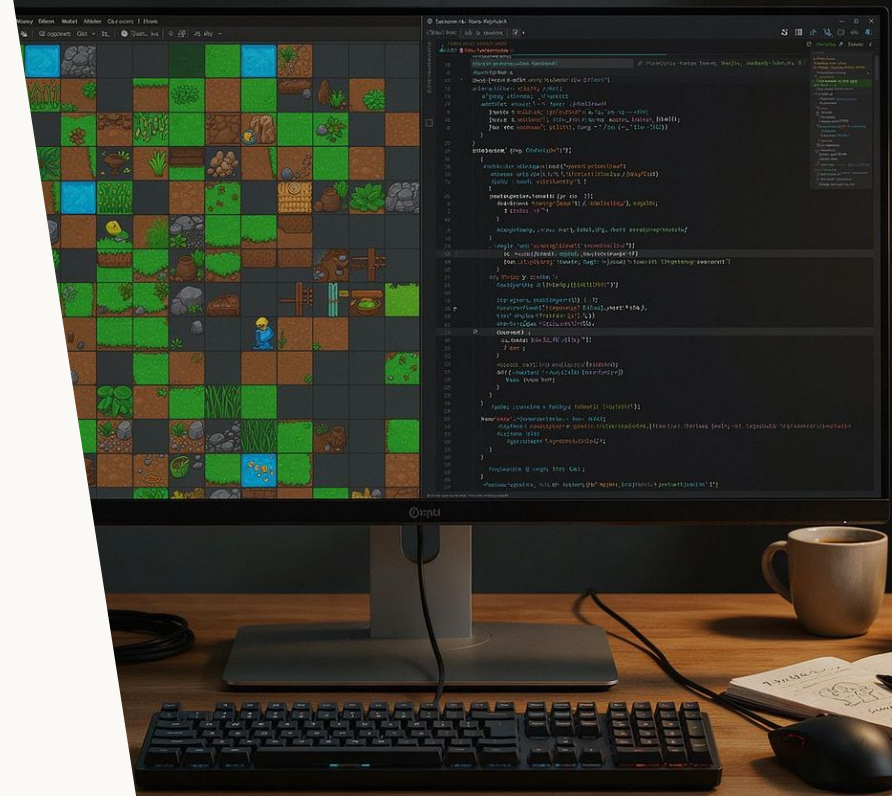
- Sprites e texturas coloridas
- Animações e partículas
- Iluminação e sombras
- Decoração e atmosfera

Camada Lógica

O que o jogo entende

- Pode andar aqui? (walkable)
- Bloqueia movimento?
- Causa dano ao jogador?
- Reduz velocidade?
- Pode ser usado por NPCs?

Uma árvore visualmente complexa, com folhas, sombra e animação de vento, pode ser simplesmente `walkable = false` para todos os sistemas internos do jogo. **A separação entre as camadas é fundamental.**



Propriedades dos Tiles

Além de um simples identificador numérico, os tiles podem carregar **propriedades estruturadas** que descrevem seu comportamento no mundo do jogo. Essa abordagem começa a transformar o mapa em algo interpretável por agentes inteligentes.

GRAMA

walkable = true
movementCost = 1

PAREDE

walkable = false
movementCost = ∞

ÁGUA

walkable = false
causeDamage = true

LAMA

walkable = true
movementCost = 2

- ❏ O campo `movementCost` será essencial nas próximas aulas, quando estudarmos algoritmos como o A* que calculam o caminho de menor custo entre dois pontos.

Coordenadas do Grid e Coordenadas do Mundo

Existem dois sistemas de coordenadas coexistindo no mesmo jogo. Saber converter entre eles é uma habilidade fundamental para implementar colisão, interação e navegação.



Coordenada do Grid

Identifica a **célula** na matriz:

coluna = 4
linha = 3

É a posição lógica — usada pela IA, pela colisão e pelas regras do jogo.



Coordenada no Mundo

Identifica o **pixel** na tela:

x = 128 px
y = 96 px

É a posição visual — usada para renderizar sprites, câmera e efeitos.

A conversão entre os dois sistemas utiliza o **tamanho do tile** (ex.: 32×32 pixels) como fator de escala:

```
worldX = coluna × tileSize // 4 × 32 = 128  
worldY = linha × tileSize // 3 × 32 = 96
```


Do Mundo para o Tile: Conversão Inversa

Dado que sabemos a posição do personagem em pixels, precisamos descobrir em qual célula do grid ele está. Essa operação é realizada com a **divisão inteira** (floor), garantindo que obtemos sempre um índice válido da matriz.

Fórmula

```
coluna = floor(x / tileSize)
```

```
linha = floor(y / tileSize)
```

Exemplo prático

```
tileSize = 32 px
```

```
x = 150, y = 70
```

```
coluna = floor(150 / 32) = 4
```

```
linha = floor(70 / 32) = 2
```

```
→ mapa[2][4]
```

Onde essa conversão é usada?

- **Colisão:** qual tile o personagem está tocando?
- **Interação:** há um objeto nessa célula?
- **Movimentação:** a próxima posição é válida?
- **IA:** onde está o jogador no grid?
- **Navegação:** qual célula é o destino?

Essa é uma das operações mais frequentes em qualquer engine de jogo baseada em tiles.

O que é Colisão?

Colisão é o mecanismo que determina se dois elementos do jogo estão **ocupando ou tentando ocupar espaços incompatíveis**. Sem colisão, personagens atravessariam paredes, projéteis ignorariam inimigos e o jogo perderia toda a coerência física.



Jogador × Parede

O movimento do personagem deve ser bloqueado ao tentar ocupar uma célula intransitável.



Projétil × Obstáculo

Balas e flechas precisam detectar colisão com paredes e inimigos para causar efeito.



Inimigo × Cenário

NPCs e agentes também não devem atravessar o mapa — a colisão afeta todos os atores.

A vantagem dos mapas em tiles: **não precisamos testar o personagem contra cada elemento do mapa**. Basta verificar as poucas células adjacentes à sua posição atual.

Colisão Baseada em Tiles

O fluxo de verificação de colisão em jogos com Tile Map é simples e eficiente: antes de mover o personagem, verificamos se a célula de destino é transitável.

Situação no mapa

```
X X X X X
X . P . X
X . . . X
X X X X X
```

X = parede · **.** = chão · **P** = jogador

O jogador tenta mover para cima — há uma parede. O movimento deve ser cancelado.

Pseudocódigo

```
novaLinha = linha + deltaLinha
novaColuna = coluna + deltaColuna

if mapa[novaLinha][novaColuna]
  .walkable then
    moverJogador()
  else
    bloquearMovimento()
end
```

Quatro etapas: **calcular** nova posição → **descobrir** o tile → **verificar** se é transitável → **permitir ou bloquear** o movimento.

Bounding Boxes e AABB

Na prática, personagens não são pontos matemáticos — eles ocupam uma área. O **AABB (Axis-Aligned Bounding Box)** é a técnica mais simples e eficiente para representar essa área como um retângulo alinhado aos eixos.

Como funciona o AABB

Cada entidade possui um retângulo invisível que define sua área de colisão. Dois retângulos colidem quando há sobreposição simultânea nos eixos X e Y:

```
colide = (A.x < B.x + B.w) AND  
         (A.x + A.w > B.x) AND  
         (A.y < B.y + B.h) AND  
         (A.y + A.h > B.y)
```

Combinando com tiles

- Verifica o tile na posição destino
- Aplica AABB para precisão fina
- O sprite visual $\neq$ área de colisão

Um personagem grande pode ter uma caixa de colisão menor que seu sprite, tornando o jogo mais justo e agradável.

- ❏ Em jogos baseados em tiles, a verificação do tile cobre a maioria dos casos. O AABB complementa para interações mais precisas entre entidades móveis.

O Mapa como Conhecimento para a IA

Retomando tudo que construímos, surge uma pergunta central para as próximas aulas:

"Se um inimigo precisa chegar até o jogador, como ele sabe onde pode andar?"

Matriz de navegabilidade

```
0 0 0 1 1
0 1 0 0 0
0 1 0 1 0
0 0 0 1 0
```

0 = transitável · **1** = bloqueado

Essa representação simples é o ponto de partida para qualquer algoritmo de navegação de agentes.

O que a IA poderá fazer com isso

- Encontrar caminhos entre dois pontos
- Calcular distâncias e rotas
- Evitar obstáculos dinamicamente
- Procurar e perseguir o jogador
- Patrulhar áreas do mapa

 **Ideia central:** Antes de ensinar o agente a pensar, precisamos ensinar o agente a compreender o espaço onde ele existe.

Atividade Prática

PROJETO DO JOGO – AULA 03

Crie ou evolua o mapa do seu projeto de jogo implementando uma estrutura completa de Tile Map com representação lógica.

✓ Requisitos obrigatórios

- Mínimo de **15 × 15 tiles**
- Pelo menos **3 tipos** de terreno
- Áreas transitáveis e não transitáveis
- Obstáculos definidos na matriz
- Posição inicial do jogador no mapa
- Personagem não atravessa tiles bloqueados
- Personagem permanece dentro dos limites

★ Desafio adicional

Crie um terreno transitável com **custo de movimentação diferenciado**:

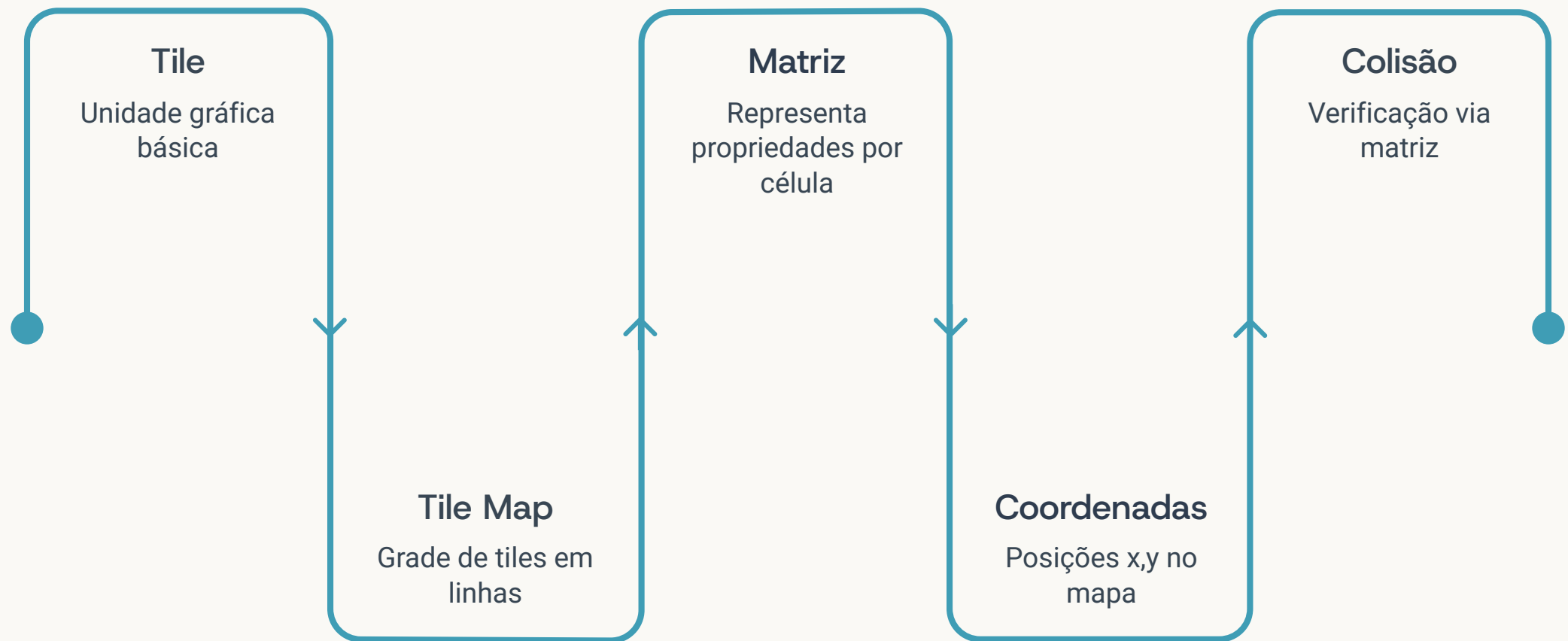
chão = custo 1

lama = custo 2

Não é necessário implementar busca de caminho agora. Apenas defina a propriedade `movementCost` na estrutura do tile.

Essa propriedade será reutilizada quando estudarmos o **algoritmo A***.

Síntese e Conexão com a Próxima Aula



Próxima Aula · Aula 04

"Se sabemos quais posições são transitáveis, como um inimigo pode encontrar o caminho até o jogador?"

Na **Aula 04**, vamos explorar algoritmos de busca em grafos — como **BFS** e **A*** — aplicados diretamente ao Tile Map. Com eles, implementaremos navegação inteligente para NPCs: inimigos que encontram rotas, desviam de obstáculos e perseguem o jogador de forma autônoma.